Stage 3: System Design - deeper level

In this stage, I will describe how requests and events move through the system, what services do on the happy path, and how the system handles distributed systems issues such as failures, duplicates, retries, ordering, caching, and scalability.

Request/Event:

User Updates notification preferences

Request content:

- userID
- Selected teams, games or leagues the user wants to follow
- Notification settings (event types the user wants alerts for)

Desired response:

- The desired response is a confirmation that the user's preferences were successfully saved and that future notifications will follow the updated settings

Happy Path:

1. The user opens the application and changes their notification preferences.
2. The application creates a preference update request containing the user ID, selected teams/games/leagues, and notification settings.
3. The application sends the request to the User Preference Service.
4. The User Preferences Service validates the request data.
5. The User Preferences Service stores the updated preferences in persistent storage.

6. After the preferences are stored successfully, the User Preferences Service updates the cached subscription mappings used by the Pub/Sub Workers.
7. The User Preference Service returns a success confirmation to the application
8. The application displays the confirmation to the user.

Possible system problems and responses:

- **Step 3: The application sends the request to the User Preference Service.**
    - **Possible problem:** The request could be delayed, lost, or timed out, due to network issues.
    - **Response:** The application should retry the request after a timeout until it succeeds.

    - **Possible problem:** The same request could be sent more than once if the user presses save multiple times or if the application retries after a network issue.
    - **Response:** The system should treat repeated requests the same way, so updating preferences multiple times with the same data does not cause problems.

- **Step 4: The User Preferences Service validates the request data.**
    - **Possible problem:** Even if the application performs validation, invalid requests may still reach the service due to client bugs, or external requests.
    - **Response:** The User Preference Service should validate all incoming requests and reject invalid request to ensure system integrity and prevent corrupted or inconsistent state.

- **Step 5: The User Preferences Service stores the updated preferences in persistent storage.**

    - **Possible problem:** The service could crash while writing the new preferences.
    - **Response:** The write should be handled atomically, so the system either fully stores the updated preferences or leaves the old preferences unchanged.

    - **Possible problem:** Two preference updates from the same user could arrive close together.
    - **Response:** The service should use versioning or last-write-wins logic so the newest update becomes the valid one.

- **Step 6: After the preferences are stored successfully, the User Preferences Service updates the cached subscription mappings used by the Pub/Sub Workers.**
    - **Possible problem:** The preferences may be stored successfully, but the cache update could fail.
    - **Response:** The database remains the source of truth, and the cache update should be retried until it succeeds.

    - **Possible problem:** The cache may briefly contain stale data
    - **Response:** The system can handle a short period of eventual consistency because this greatly improves scalability during live game events.

- **Step 7: The User Preference Service returns a success confirmation to the application**
    - **Possible problem:** The preferences may be stored, but the confirmation message may fail.

- **Response:** If the application retries the request, the service should safely return the completed result without incorrectly applying the update multiple times.

Live game event triggers notifications

Request content:

- gameID
- eventID
- eventType
- Team involved in the event
- Timestamp
- Event description

Desired response:

- The desired response is that the event is successfully processed, the current game state is updated, and notifications are sent to all the users whose preferences match the event.

Happy path

1. The Sports Data Provider sends a live game event to the Event Ingestion Service.
2. The Event Ingestion Service receives the event.
3. The Event Ingestion Service normalizes the event data into the format used by the system.
4. The Event Processing Service processes the event.
5. The Event Processing Service updates the Game State Management Service so the system reflects the newest score/status.
6. The Event Processing Service publishes the event to the Pub/Sub Broker.
7. The Pub/Sub Broker queues the event for processing.

8. A Pub/Sub Worker consumes the event from the broker.
9. The Pub/Sub Worker uses cached subscription mappings to determine which users should receive notifications for that event.
10. The Pub/Sub Worker sends notification jobs to the Notification Service.
11. The Notification Service builds the notification payload.
12. The Notification Service stores the notification in notification history.
13. The Notification Service sends the notification request to the Push Notification Service.
14. The Push Notification Service delivers the notification to the user's device.
15. The user receives the notification.

Possible system problems and responses:

- **Step 1: The Sports Data Provider sends a live game event to the Event Ingestion Service.**
    - **Possible problem:** The provider connection could temporarily fail, causing live events to stop arriving.
    - **Response:** The Event Ingestion Service should retry the connection until the provider becomes available again.

- **Step 2:** The Event Ingestion Service receives the event.
    - **Possible problem:** The service may go down right after receiving the event.
    - **Response:** The system should store events durably so they are not lost if a service crashes after receiving them.

- **Step 3:** The Event Ingestion Service normalizes the event data into the format used by the system.
    - **Possible problem:** The incoming event may be missing fields or can even be malicious.
    - **Response:** The Event Ingestion Service should validate the event format and reject invalid events instead of forwarding corrupted data through the system.

- **Step 4:** The Event Processing Service processes the event.
    - **Possible problem:** Events for the same game may arrive out of order. For example, an earlier play might arrive after a later score update.
    - **Response:** Service should use timestamps or a sequence counter and ignore old events that would overwrite the newer game state.

    - **Possible problem:** The same event may be processed more than once.
    - **Response:** Event processing should safely handle duplicate events so the final result stays correct even if the same event is seen again

- **Step 5:** The Event Processing Service updates the Game State Management Service so the system reflects the newest score/status.
    - **Possible problem:** Two updates for the same game may happen at nearly the same time.
    - **Response:** Updates for the same game should be applied in order, or the system should use version checks so older updates do not overwrite the newer ones.

    - **Possible problem:** The game state update may fail even though the event was already received.
    - **Response:** The system should retry the state update and avoid marking the event fully complete until the update succeeds.

- **Step 6:** The Event Processing Service publishes the event to the Pub/Sub Broker.
    - **Possible problem:** The Pub/Sub Broker could be temporarily unavailable.
    - **Response:** The Event Processing Service should retry publishing the event until it is accepted by the broker.

    - **Possible problem:** The event may be published successfully, but the confirmation may not be received.

    - **Response:** The publish operation should safely handle retries so the same event is not incorrectly treated as a brand new event each time.

- **Step 7:** The Pub/Sub Broker queues the event for processing.
    - **Possible problem:** A sudden increase in traffic during a major game could cause a queue of events to build up in the broker
    - **Response:** The broker should support buffering, and the system can scale by adding more Pub/Sub workers in order to process events faster off the queue.

- **Step 8:** A Pub/Sub Worker consumes the event from the broker.
    - **Possible problem:** A worker may crash while handling the event.
    - **Response:** The broker should allow the event to be delivered again to another worker so processing can continue.

- **Step 10:** The Pub/Sub Worker sends notification jobs to the Notification Service.
    - **Possible problem:** The same notification job could be sent more than once.
    - **Response:** The Notification Service should track notifications by event ID and user so duplicate jobs do not cause the same user to receive the same notification more than once.
    -

- **Step 11:** The Notification Service builds the notification payload.
    - **Possible problem:** Notification payload creation may fail because event data is incomplete or invalid.
    - **Response: t**he service should reject bad payloads instead of sending broken notifications.

- **Step 13:** The Notification Service sends the notification request to the Push Notification Service.
    - **Possible problem:** The push provider may reject the request or be temporarily unavailable.

    - **Response:** The Notification Service should retry failed push requests and keep the notification in history even if immediate delivery fails.

This flow is the most important part of the system because it must handle live events quickly and at large scale. During major games, many events may arrive at once and many users may be subscribed. This is why the system must handle failures, duplicates, stale cache data, worker crashes, push delivery problems, and traffic spikes without slowing down or sending incorrect notifications. This is why the design uses Pub/Sub, cached subscription mappings, retries, durable event handling, and scalable workers.

User requests current game state

Request content:

- userID
- gameID

Desired response:

- The user receives the most up-to-date game state, including the current score and game status.

Happy Path:

1. The user opens the application.
2. The user selects a game to view.
3. The application creates a request containing the user ID and game ID.
4. The application sends the request to the Game State Management Service.
5. The Game State Management Service retrieves the current game state.
6. The Game State Management Service returns the game state to the application.
7. The application displays the current score and game status to the user.

Possible system problems and responses:

- **Step 4:** The application sends the request to the Game State Management Service.
    - **Possible problem:** The request may be delayed or time out due to network issues.
    - **Response:** The application should retry the request or display the last known game state if available.

- **Step 5:** The Game State Management Service retrieves the current game state.
    - **Possible problem:** The game state may be slightly outdated if the latest event has not been processed yet.
    - **Response:** The system accepts a small delay in updates (eventual consistency) since near real-time updates are sufficient for this use case.

    - **Possible problem:** The service may be under heavy load during a popular game.
    - **Response:** The system can scale by adding more instances of the Game State Management Service and caching frequently requested game states.

- **Step 6:** The Game State Management Service returns the game state to the application.
    - **Possible problem:** The response may be lost before reaching the application.
    - **Response:** The application can safely retry the request since retrieving game state is a read operation and does not change system data.

This request represents the read path of the system. It must handle many users requesting the same game at the same time, especially during popular events. The system must support high read traffic, tolerate slight delays, and remain responsive even under heavy load. This is achieved through caching, scaling, and allowing eventual consistency.

Diagram-6-Event Processing Service State Chart

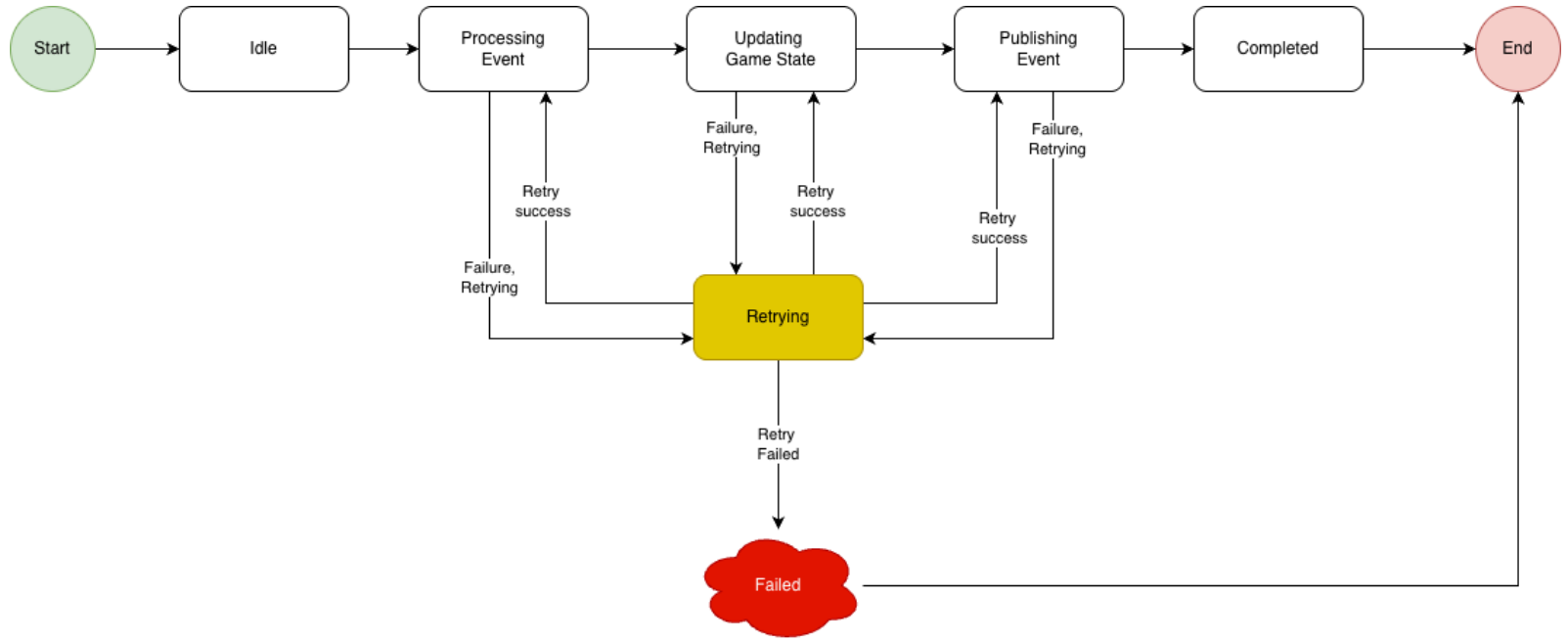


Diagram-7-Pub:Sub Worker State Chart

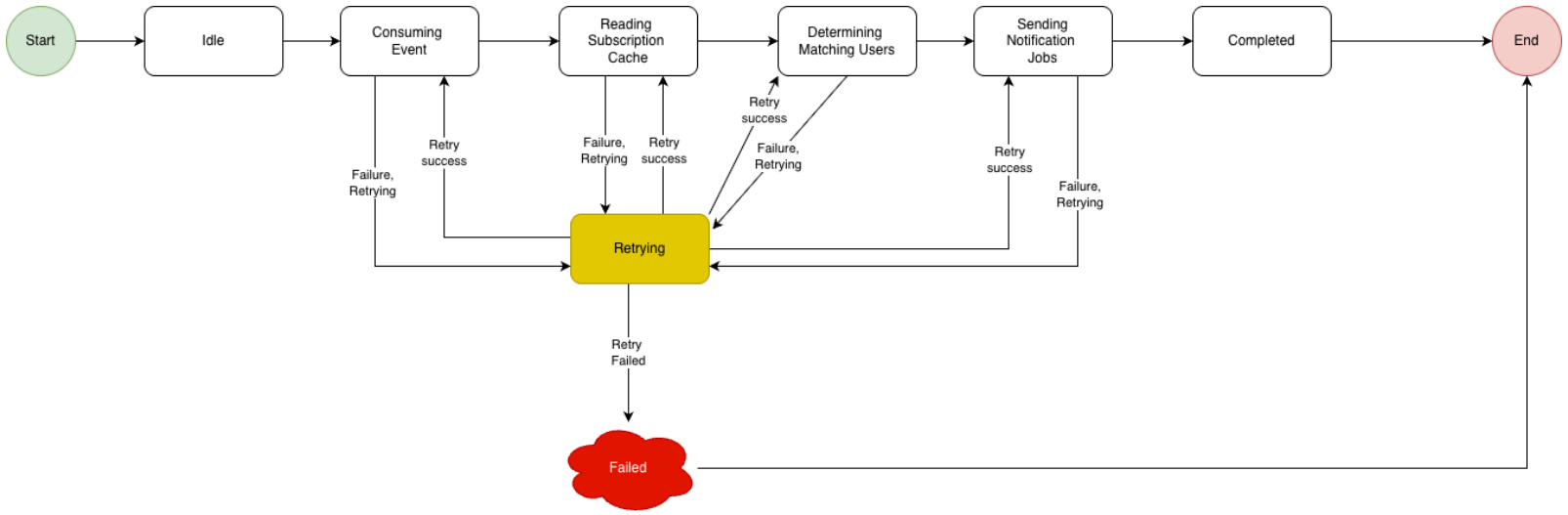


Diagram-8-User Preference Service State Chart

